



# Instruções de Uso do Verificador de Assinaturas (IUVA)

**! CLASSIFICAÇÃO: CONFIDENCIAL – USO INTERNO ESTRITO**

**Versão do Documento:** 2.2.0 (*Atualização de Segurança*)

**Ferramenta Alvo:** verificador

---

## 1. Visão Geral do Sistema

O **Verificador de Assinaturas** é a ferramenta central de validação de comunicação interna do nosso grupo. Para garantir a autenticidade, o não-repúdio e a integridade das ordens emitidas pelos líderes da operação, todas as mensagens devem passar por uma dupla camada de verificação (Simétrica e Assimétrica) antes de qualquer execução.

Este documento estabelece as diretrizes técnicas de funcionamento do **verificador** e as regras de gestão da nossa infraestrutura de chaves (PKI) descentralizada.

## 2. Arquitetura de Confiança e Verificação

Devido à necessidade de operarmos em ambientes isolados (*offline/air-gapped*), nosso verificador não se comunica com servidores externos ou Autoridades Certificadoras (CAs) globais.

Adotamos um modelo de **Confiança Baseada em Diretório Local**. A aprovação de uma assinatura depende exclusivamente de o certificado público do emissor estar previamente catalogado e autorizado na infraestrutura da máquina onde a ferramenta está sendo executada.

### O Processo de Validação (v2.2.0)

Quando o **verificador** é executado, ele exige quatro componentes e realiza as seguintes etapas em milissegundos:

| Passo | Camada de Segurança | Ação do Verificador |
|-------|---------------------|---------------------|
|       |                     |                     |

|    |                     |                                                                                                                                                                                       |
|----|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1º | Integridade Rápida  | Calcula localmente o <b>HMAC-SHA256</b> do arquivo da mensagem usando a chave secreta embutida e compara com o <i>hash</i> fornecido. Se falhar, a operação é imediatamente abortada. |
| 2º | Cadeia de Confiança | Varre a pasta <code>./trust_store/</code> em busca de uma Autoridade Certificadora (CA) raiz que reconheça e valide o certificado do operador.                                        |
| 3º | Extração            | Extrai a chave pública do certificado do operador previamente validado.                                                                                                               |
| 4º | Autenticidade       | Verifica se a assinatura criptográfica ( <code>.bin</code> ) condiz com a chave pública extraída e com o arquivo original da mensagem.                                                |

### **NOTA TÉCNICA DE SEGURANÇA (NOVO):**

Devido às tentativas recentes de corrupção de dados na rede por adversários, todas as ordens agora devem obrigatoriamente acompanhar um código de integridade **HMAC-SHA256** gerado com a chave secreta de sistema.

---

## 3. Detalhes Técnicos da Verificação

Para fins de auditoria interna e manutenção pelos operadores, abaixo detalhamos o trecho exato do código-fonte responsável por validar a dupla camada (Integridade + Autenticação). O sistema realiza verificações em memória e chamadas diretas à biblioteca OpenSSL do sistema operacional.

```

# snippet_validacao.py - Núcleo de Processamento do Verificador v2.2.0

import os

import sys

import subprocess

import hmac

import hashlib


def validar_ordem_interna(arquivo_mensagem,      arquivo_assinatura,
nome_certificado_assinante, hmac_fornecido):

    diretorio_confianca = "./.trust_store/"

    caminho_cert_assinante = f"./{nome_certificado_assinante}"

    print("=====")
    print(" VERIFICADOR DE ASSINATURAS E CONFIANÇA - v2.2.0 ")
    print("=====")

    # =====
    # PASSO 1: Verificação de Integridade (HMAC)
    # =====

    print("[PASSO 1] Verificando Integridade...")

    try:

        with open(arquivo_mensagem, 'rb') as f:

            mensagem_bytes = f.read()

            hmac_calculado = hmac.new(b"1nt3gr1d4d3",      mensagem_bytes,
            hashlib.sha256).hexdigest()

            if not hmac.compare_digest(hmac_calculado, hmac_fornecido):

                print("[ERRO FATAL] Integridade violada ou código HMAC
incorreto. Abortando.")

                sys.exit(1)

```

```

        print("[SUCESSO] Código HMAC coincide. O arquivo não foi
alterado.")

    except Exception as e:

        print(f"[ERRO] Falha ao processar o arquivo para HMAC: {e}")

        sys.exit(1)

# =====

# PASSO 2: Verificação da Cadeia de Confiança

# =====

if not os.path.exists(caminho_cert_assinante):

    print(f"[ERRO] Certificado '{nome_certificado_assinante}' não
encontrado no diretório atual.")

    sys.exit(1)

print("\n[PASSO 2] Verificando Cadeia de Confiança do
Certificado...")

ca_confivel_encontrada = False

if os.path.exists(diretorio_confianca):

    for ca_file in os.listdir(diretorio_confianca):

        caminho_ca = os.path.join(diretorio_confianca, ca_file)

        if os.path.isfile(caminho_ca):

            # Tenta validar o certificado do assinante contra a CA
atual

            comando_verify = f"openssl verify -CAfile {caminho_ca}
{caminho_cert_assinante}"

            res_verify = subprocess.run(comando_verify, shell=True,
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

            if res_verify.returncode == 0:

```

```

        ca_confiavel_encontrada = True

        break

    if not ca_confiavel_encontrada:

        print("[ERRO] Operação Negada: A cadeia de confiança do
certificado não pôde ser validada por nenhuma CA local.")

        sys.exit(1)

    # =====

    # PASSO 3 e 4: Extração e Verificação

    # =====

    print("\n[PASSO 3] Confiança estabelecida. Extraíndo chave pública
para verificação da assinatura...")

    comando_pubkey = f"openssl x509 -pubkey -noout -in
{caminho_cert_assinante}"

    chave_publica = subprocess.run(comando_pubkey, shell=True,
capture_output=True, text=True).stdout

    with open("temp_pubkey.pem", "w") as f:

        f.write(chave_publica)

    print("[PASSO 4] Verificando Assinatura Digital ...")

    comando_dgst = f"openssl dgst -sha256 -verify temp_pubkey.pem
-signature {arquivo_assinatura} {arquivo_mensagem}"

    try:

        resultado_dgst = subprocess.run(comando_dgst, shell=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)

        if resultado_dgst.returncode == 0:

            print("\n[SUCESSO] Assinatura Válida. Ordem autêntica e

```

```
pronta para execução.")

    print("Flag: CTF{Pwned_Trust_Chain_And_HMAC_Bypass}")

else:

    print("\n[FALHA] A assinatura criptográfica não confere com o arquivo original.")

except Exception as e:

    print("[ERRO FATAL] Falha na execução do processo openssl.")

finally:

    if os.path.exists("temp_pubkey.pem"):

        os.remove("temp_pubkey.pem")
```

---

## 4. Gestão de Autoridades Certificadoras (CAs)

A arquitetura do `verificador` foi projetada para ser ágil e de fácil atualização pelos operadores em campo, eliminando a necessidade de recompilar o código-fonte a cada mudança de liderança ou rotação de chaves.

### Como autorizar uma nova entidade (Adicionar nova CA):

Se for necessário conceder permissão para que um novo operador emita ordens válidas, siga rigorosamente os passos abaixo:

1. **Geração da Chave:** O novo operador deve gerar seu par de chaves e seu certificado público (formato `.pem`).
2. **Inclusão no Repositório:** O operador responsável pela máquina deve copiar o arquivo `.pem` da nova CA e colá-lo diretamente na pasta `./trust_store/`, localizada no mesmo diretório do executável.
3. **Efeito Imediato:** Não há necessidade de reiniciar serviços ou alterar configurações de banco de dados. A partir do momento em que o certificado `.pem` estiver fisicamente presente na pasta `./trust_store/`, o `verificador.exe` o assumirá como uma raiz de confiança absoluta.

